Agile Mindset

What is Agile

- Developed for Software projects, but it is a methodology that can be used on all Projects types
- Agile is an umbrella term that is used to refer to different types of iterative development
- Scrum is the most common method of agile, there are others such as extreme programming (XP), lean development, and Kanban.

Characteristics of four life cycles

Characteristics				
Approach	**Requirements**	**Activities**	**Delivery**	**Goal**
Predictive	Fixed	Performed once for the entire project	Single delivery	Manage cost
Iterative	Dynamic	Repeated until correct	Single delivery	Correctness of solution
Incremental	Dynamic	Performed once for a given increment	Frequent smaller deliveries	Speed

Agile leverage both aspect of Iterative and incremental

PMI Agile Practice Guide, Page 18 table 3.1

Agile Benefits

- Customer involved throughout the life cycle
- Greater Customer Interaction with all stakeholders
- Constant Feedback is required to stay current and successful
- Greater Value up front
- Change is welcomed by all stakeholders

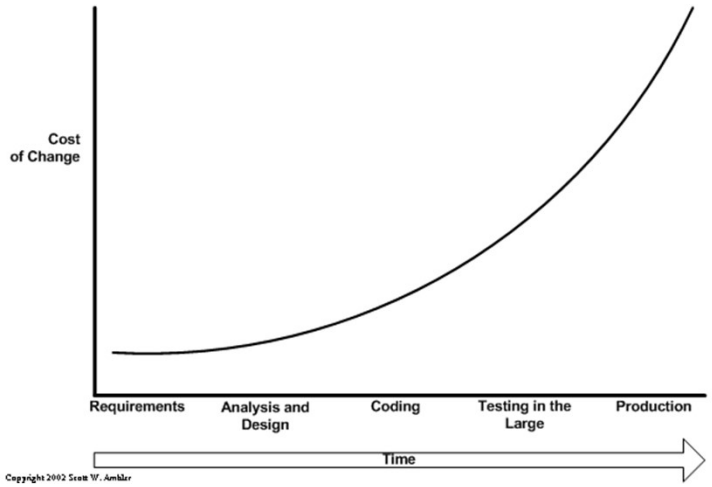
What is Value

- "The worth, importance, or usefulness of something.“
- -PMI
- In the context of **projects and project management**, value refers to the **benefits delivered in relation to the effort and cost**.
- It can be tangible (e.g., revenue, cost savings) or intangible (e.g., customer satisfaction, brand reputation), and it should align with the **organization’s strategic objectives**.

Delivering Value Incrementally

- Incremental delivery is about deploying working parts of a product over the life of the project
- In software development, its first delivered to a testing environment then to production
- This will reduce the amount of rework by discovering issues early and fixing them

Delivering Value Incrementally



http://www.agilemodeling.com/essays/costOfChange.htm

Agile ***Declaration of Interdependence (DOI)***

- *Agile and adaptive approaches for linking people, projects and value*
- *We are a community of project leaders that are highly successful at delivering results. To achieve these results:*
- *We **increase return on investment** by making continuous flow of value our focus.*
- *We **deliver reliable results** by engaging customers in frequent interactions and shared ownership.*
- *We **expect uncertainty** and manage for it through iterations, anticipation, and adaptation.*
- *We **unleash creativity and innovation** by recognizing that individuals are the ultimate source of value, and creating an environment where they can make a difference.*
- *We **boost performance** through group accountability for results and shared responsibility for team effectiveness.*
- *We **improve effectiveness and reliability** through situationally specific strategies, processes and practices.*

©2005 David Anderson, Sanjiv Augustine, Christopher Avery, Alistair Cockburn, Mike Cohn, Doug DeCarlo, Donna Fitzgerald, Jim Highsmith, Ole Jepsen, Lowell Lindstrom, Todd Little, Kent McDonald, Pollyanna Pixton, Preston Smith and Robert Wysocki.

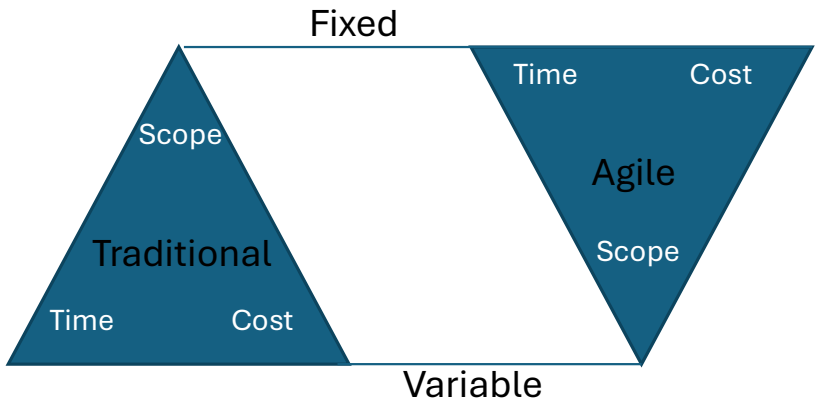
Agile Mindset

- Welcoming change
- Working in small value increments
- Using build and feedback loops
- Learning through discovery
- Value-driven development
- Failing fast with learning
- Continuous delivery
- Continuous improvement

Agile vs. Traditional Project Management

- Agile builds in increments vs. as a whole
- Agile does planning throughout vs. done all at once
- Agile delivers products over time vs. all at once
- Customers sees value faster vs. at the end
- Agile wants changes vs. discouraging changes

Inverting the Triangle



Agile Manifesto

- Create in 2001
- Contains:
  - 4 values
  - 12 guiding principles

https://agilemanifesto.org/

The Agile Manifesto Values

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

Individuals & interactions	**over**	Processes & tools
Working software	**over**	Comprehensive documentation
Customer collaboration	**over**	Contract negotiation
Responding to change	**over**	Following a plan

That is, while there is value in the items on the **right**, we value the items on the **left** more.

www.agilemanifesto.org

Individuals and interactions over processes and tools

- While processes and tools will likely be necessary on our projects, we should focus the team's attention on the individuals and interactions involved.
- Projects are undertaken by people, not tools
- Problems get solved by people, not processes
- Projects are ultimately about people

Working software over comprehensive documentation

- Focus on the delivering value vs. paperwork.
- Agile documents should be barely sufficient
- Done just in time
- Done just because
- Delivering software that does what it should comes first, before creating documentation.
- Agile dramatically simplify the administrative paperwork relating to time, cost, and scope control

Customer collaboration over contract negotiation

- Be flexible and accommodating, instead of fixed and uncooperative
- Manage change, don't suppress change
- Shared definition of "done"
- Requires trusting relationship

Responding to change over following a plan

- Spend effort and energy responding to changes
- Software projects tend to have high rates of change

Agile Guiding Principles 1-3

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

Agile Guiding Principles 4-6

4. Business people and developers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

Agile Guiding Principles 7-9

7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers)and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.

Agile Guiding Principles 10-12

10. Simplicity; the art of maximizing the amount of work not done is essential.
11. The best architectures, requirements and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects on how to become more effective then tunes and adjusts its behavior accordingly.

Agile Methods

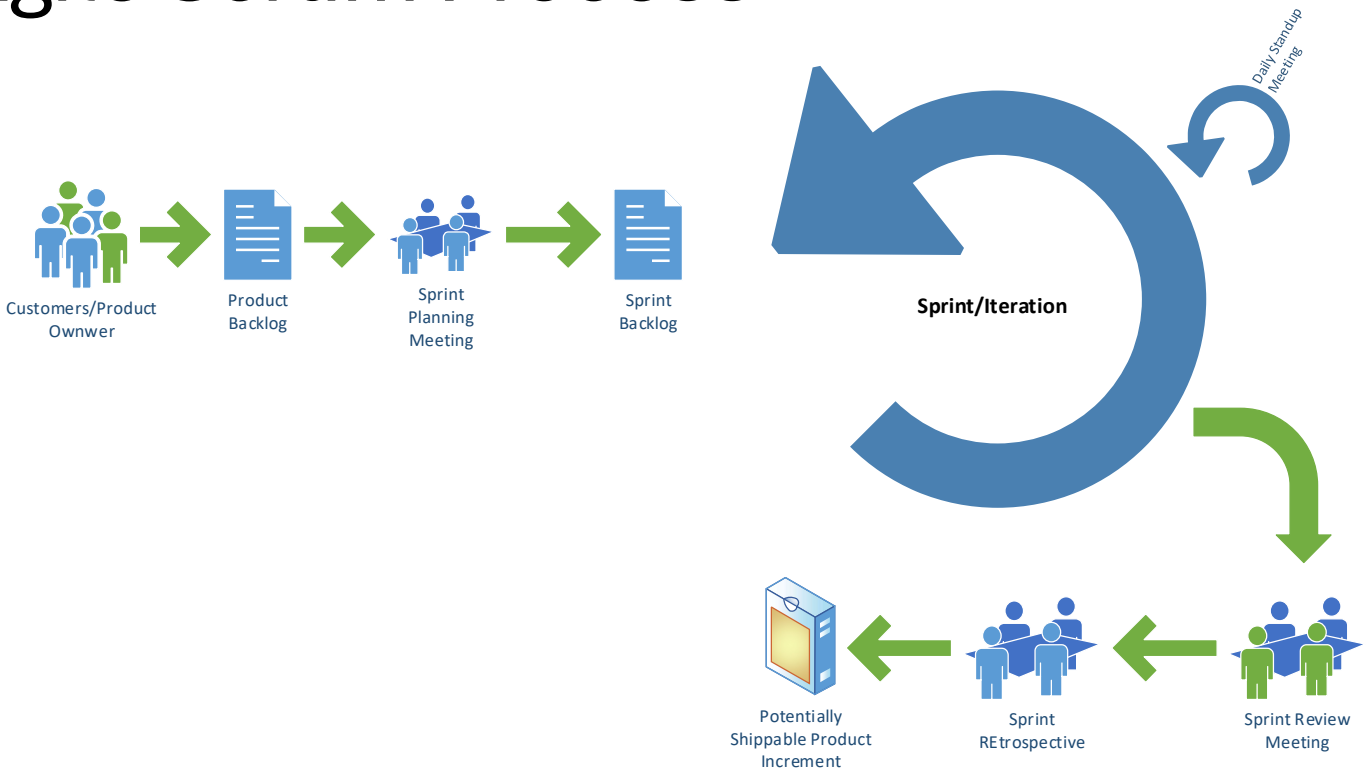
- Many agile methodologies
- Scrum
- Extreme Programming (XP)
- Kanban Development
- Lean Software Development
- Scaled Agile (SAFe)

Agile Scrum Terms

- **Product Owner -** Designated person that represents the customer on the project
- **Agile Project Manager/Scrum Master –** Manages the agile project
- **Product Backlog -** Project requirements from the stakeholders
- **Sprint Planning Meeting-** Meeting done by the agile team to determine what features will be done in the next sprint
- **Sprint Backlog –** Work the team selects to get done in the next sprint
- **Sprint -** A short iteration where the project teams work to complete the work in the sprint backlog, (1-4 weeks typical)
- **Daily Stand Up Meeting -** A quick meeting each day to discuss project statuses, led by the Scrum Master. Usually 15 minutes
- **Sprint Review –** An inspection done at the end of the sprint by the customers
- **Retrospective –** Meeting done to determine what went wrong during the sprint and what when right. Lesson learned for the sprint.
- **Partial Completed Product -** Customers Demo the product and provides feedback. This feedback adjust the next Sprint priorities
- **Release -** Several Sprints worth of work directed to operations for possible rollout and testing

Sprint = Iteration

Agile Scrum Process



Scrum

- Set of team guidance practices, roles, events, artifacts, and rules
- Based on three pillars of Transparency, Inspection, and Adaptation:
  - Transparency
    - Visibility to those responsible for the outcome
  - Inspection
    - Timely checks on how well a project is progressing toward goals
    - Looks for problematic deviations or differences from goals
  - Adaptation
    - Adjusting a process to minimize further issues if an inspection shows a problem or undesirable trend

Scrum Roles & Responsibilities

Product Owner

- Owns Product vision
- Defines features, decides on release date and content
- Responsible for market success
- Prioritizes features according to market value
- Can change features and priorities every Sprint

ScrumMaster

- Responsible for facilitating process
- Focuses Team and protects them from external interruption
- Looks for ways to enhance productivity
- Assists Product Owner in leveraging Scrum

Scrum Roles & Responsibilities

Development Team

- Small group containing all necessary project skills
- Focuses on steady delivery of high quality features
- Generates options for delivery
- Manages own work within Sprints

Scrum Activities

The Scrum methodology refers to several different types of activities:

1. sprint planning meeting
2. sprints
   - Daily stand-up meeting
3. sprint review meeting
4. sprint retrospectives.

Sprint Planning Meeting

- ✓ Used to determine what work will be done in that sprint and how the work will be achieved.
- ✓ The development team predicts what can be delivered based on estimates, projected capacity, and past performance to define the sprint goal.
- ✓ The development team then determines how this functionality will be built and how the team will organize to deliver the sprint goal.
- ✓ Output of this will be the sprint backlog. The work to get done in the next sprint.

Sprints

- ✓ A sprint is a **timeboxed** (time-limited) iteration of 1-4 weeks to build a potentially releasable product
- ✓ Each sprint includes a sprint planning meeting, daily Scrum, the actual work, a sprint review meeting, and the sprint retrospective
- ✓ During the sprint, no changes are made that would affect the sprint
- ✓ The development team members are kept the same throughout the sprint

Daily Scrum (or Standup)

- ✓A 15-minute time-boxed activity for the Development Team to synchronize activities and create a plan for the next 24 hours
- ✓Should be held at the same time and place each day
- ✓Each team member should answer 3 questions:
  1. What did you do yesterday?
  2. What will you do today?
  3. Are there any impediments in your way?

Sprint Review

- ✓Takes place at the end of the Sprint
- ✓Designed to gather feedback from stakeholders on what the Team has completed in the sprint
- ✓Team demonstrates work that was completed during the sprint
- ✓To create a conversation between the Team and the stakeholders about how to make the product better
- ✓should be time boxed to no more than an hour per week of Sprint

Sprint Retrospective

- ✓Opportunity for the Team to inspect and create a plan for improvements to be done during the next Sprint.
- ✓Team discusses:
  - ✓What went well
  - ✓What went wrong
  - ✓What to do more of
  - ✓What to do less of

Scrum Artifacts

- ✓Product increment
  - ✓Part of the product that is complete after each sprint
- ✓Product Backlog
  - ✓Prioritized list of valuable items to deliver
- ✓Sprint Backlog
  - ✓List of committed items to be addressed within Sprint

Product Backlog

- ✓Prioritized list of all work that needs to be done to complete the product
- ✓List is dynamics, it evolves as the more work is added and prioritized
- ✓Items in it is prioritized by the product owner and is sorted by value
- ✓Most valuable items are listed first
- ✓Constantly being refined as more work is added to it.
- ✓Team and product owner will “groom the backlog”.

Product Increment

- ✓Part of the product that is done after each sprint
- ✓Done to get feedback after each sprint
- ✓The product owner and team needs to agree upon the “definition of done” before the team starts working on the product

Sprint Backlog

- ✓The sprint backlog is the set of items from the product backlog that were selected for a specific sprint.
- ✓The sprint backlog is accompanied by a plan of how to achieve the sprint goal, so it serves as the development team's forecast for the functionality that will be part of the sprint.
- ✓It is a highly visible view of the work being undertaken and may only be updated by the development team.

Definition of Done (DoD)

- Definition of Done (DoD) is a shared understanding of what it means when work is considered done, it should be defined at the beginning of the project, and it applies globally to the project.
- Definition of Done (DoD) is a crucial element of a successful scrum software development
- Might include things such as:
  - DoD for Unit & functional tests.
  - DoD Documentation.
  - DoD for a Writing code.

Extreme Programming (XP)

- Software development centric agile method
- Focus software development good practices
- Scrum at the project management level focuses on prioritizing work and getting feedback

XP Core Values

- Simplicity
  - Reduce complexity, extra features, and waste
  - “ Find the simplest thing that could possibly work"
- Communication
  - Team members know what is expected of them and what other people are working on
  - Daily stand-up meeting is key communication component
- Feedback
  - Get impressions of correctness early
  - Failing fast allows for faster improvement

XP Core Values

- Courage
  - Allow our work to be entirely visible to others
- Respect
  - People work together as a team and everyone is accountable for the success or failure of the project
  - Recognize people work differently and respect those differences

XP Roles

- Coach
  - Acts as a mentor, guiding the process and helping the team stay on track. Is a facilitator helping the team become effective.
- Customer:
  - Business representative who provides the requirements, priorities, and drives the business direction for the project.
- Programmers
  - Developers who build the product. Writes the codes.
- Testers
  - Helps the customer define and write the acceptance tests for the user stories.

ProductOwner and Customer are equivalent

ScrumMaster and Coach are equivalent

XP Practices

- Planning Activities (Games):
  - Release Planning:
    - Push of new functionality all the way to the production user
    - Customer outlines the functionality required
    - Developers estimate difficult build
  - Iteration Planning:
    - Short development cycles within a release (Scrum calls "sprints")
    - Conducted at start of every iteration, or every two weeks
    - Customer explains functionality they would like in iteration
    - Developers break functionality into tasks and estimate work
    - Based on estimates and amount of work accomplished in previous iteration,

XP Practices

- Small Releases:
  - Frequent, small releases to test environments
  - Demonstrate progress and increase visibility for the customer
  - Quality is maintained: Rigorous testing or Continuous integration
- Customer Tests:
  - Customer describes one or more tests to show software is working
  - Team builds automated tests to prove software is working.
- Collective Code Ownership:
  - Any pair of developers can improve or amend any code
  - Multiple people work on all code, which results in increased visibility and knowledge of code base
  - Leads to a higher level of quality; with more people looking at the code, there is a greater chance defects will be discovered.
  - Less risk if programmer leaves, since knowledge is shared

XP Practices

- Code Standards:
  - Follow consistent coding standard
  - Code looks as if it has been written by a single, knowledgeable programmer
- Sustainable Pace:
  - While periods of overtime might be necessary, repeated long hours of work are unsustainable and counterproductive
  - The practice of maintaining a sustainable pace of development optimizes the delivery of long-term value

XP Practices

- Metaphor:
  - XP uses metaphors and similes to explain designs and create a shared technical vision.
  - These descriptions establish comparisons that all the stakeholders can understand to help explain how the system should work.
  - For example, “The invoicing module is like an Accounts receivable personnel who makes sure money collected from our customers”.
- Continuous Integration:
  - Integration involves bringing the code together and making sure it all compiles and works together.
  - This practice is critical, because it brings problems to the surface before more code is built on top of faulty or incompatible designs.

XP Practices

- Test-Driven Development (TDD):
  - The team writes tests prior to developing the new code.
  - If the tests are working correctly, the initial code that is entered will fail the tests
  - The code will pass the test once it is written correctly.
- Pair Programming:
  - In XP, production code is written by two developers working as a pair to write and provide real-time reviews of the software as it emerges.
  - Working in pairs also helps spread knowledge about the system through the team.

XP Practices

- Simple Design:
  - Code is always testable, browsable, understandable, and explainable
  - Do the simplest thing that could possibly work next. Complex design is replaced with simpler design
  - The best architectures, requirements, and designs emerge from self-organizing teams
- Refactoring:
  - Remove redundancy, eliminate unused functionality, and rejuvenate obsolete designs
  - Refactoring throughout the entire project life cycle saves time and increases quality
  - Code is kept clean and concise so it is easier to understand,
    modify, and extend

Some Basic Terminology Review

Scrum	Extreme Programming (XP)	Definition
Sprint	Iteration	Fixed-length period of time (timebox)
Release	Small Release	Release to production
Sprint/Release Planning	Planning Game	Agile planning meetings
Product Owner	Customer	Business representative to project
Retrospective	Reflection	“Lessons learned”-style meeting
ScrumMaster	Coach	Agile project manager
Development Team	Team	Empowered Cross-Functional team
Daily Scrum	Daily Standup	Brief daily status meeting

Lean Software Development

- Lean was started by Toyota as manufacturing method that was applied to software development.
- Principles:
  - Using visual management tools
  - Identifying customer-defined value
  - Building in learning and continuous improvement

Lean Software Development



Lean Software Development

- **Eliminate waste:**
  - To maximize value, we must minimize waste. For software systems, waste can take the form of **partially done work**, **delays**, **handoffs, unnecessary features.**
- **Empower the team:**
  - Rather than taking a micro-management approach, we should respect team member's superior knowledge of the technical steps required on the project and let them
- **Deliver fast:**
  - Quickly delivering valuable software and iterating through designs.
- **Optimize the whole:**
  - We aim to see the system as more than the sum of its parts.

Lean Software Development

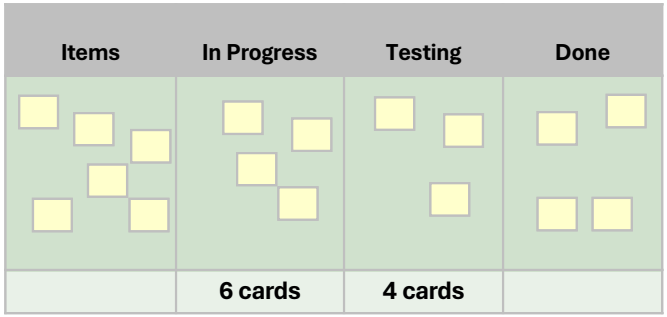
- **Build quality in:**
  - Build quality into the product and continually assure quality throughout the development process
- **Defer decisions:**
  - Balance early planning with making decisions and committing to things as late as possible.
- **Amplify learning:**
  - This concept involves facilitating communication early and often, getting feedback as soon as possible, and building on what we learn.

Seven Wastes of Lean

1. Partially done work
2. Extra Processes
3. Extra features
4. Task switching
5. Waiting
6. Motion
7. Defects

Kanban Development

- Kanban development is derived from the lean production system used at Toyota.
- "**Kanban**" is a Japanese word meaning "**signboard**."



Kanban five core principles:

- **Visualize the workflow:**
  - Software projects, by definition, manipulate knowledge, which is intangible and invisible.
- **Limit WIP:**
  - Keeping the amount of work in progress low increases the visibility of issues and bottlenecks
- **Manage flow:**
  - By tracking the flow of work through a system, issues can be identified and changes can be measured for effectiveness
- **Make process policies explicit:**
  - It is important to clearly explain how things work so the team can have open discussions about improvements
- **Improve collaboration:**
  - Through scientific measurement and experimentation, the team should collectively own and improve the processes it uses.

Feature-Driven Development (FDD)

- An Agile and model-driven methodology
- Focuses on designing & building features (small, client-valued functions)
- Best for larger teams and complex systems
- *“Feature” = small, useful result valued by the client (e.g., “Calculate invoice total”)*

Feature-Driven Development (FDD)

- The 5 Key Processes of FDD
  1. Develop Overall Model
  2. Build Feature List
  3. Plan by Feature
  4. Design by Feature
  5. Build by Feature

Each process builds upon the last — it's highly structured but still Agile.

Feature-Driven Development (FDD)

- **Develop Overall Model**
  - Understand the domain and build a high-level object model
  - Workshops with domain experts to define core concepts and relationships
  - Foundation for all future design decisions
- **Build Feature List**
  - Break down the model into a hierarchical list of features
  - Features grouped into major feature sets and subject areas
  - Each feature represents client-valued functionality
- **Plan by Feature**
  - Create a development schedule based on the feature list
  - Assign ownership of features and classes to team members
  - Allows tracking and resource planning at a granular level
- **Design by Feature**
  - Small teams design each feature in detail
  - Produce sequence diagrams and design packages
  - Led by the Chief Programmer with input from Class Owners
- **Build by Feature**
  - Code, test, and integrate the feature
  - Involves unit testing and code inspections before completion
  - Features are built in short iterations and frequently delivered

Feature-Driven Development (FDD)

- When to Use FDD **or not**
  - Large, complex systems
  - Teams needing structure and upfront modeling
  - Projects requiring **predictability and strong design discipline**
  - Not ideal for very small teams or early-stage MVP work
  - *FDD blends Agile principles with structured engineering discipline.*

Crystal Framework

- Crystal is a family of lightweight Agile methodologies
- Believes one size does not fit all — method adapts based on project size and criticality
- Focuses on people, interactions, community, skills, and communications
- Crystal values individuals over processes — like Agile Manifesto in action.

Crystal Framework

- Key Principles
  - Frequent delivery of usable code
  - Reflective improvement (retrospectives)
  - Team collaboration
  - Personal safety and team trust

Crystal Framework

Method	Team size	Project size
Crystal Clear	6 people or less	Small projects
Crystal Yellow	7–20 people	Small to medium projects
Crystal Orange	20–40 people	Medium projects
Crystal Red	40–80 people	Medium to large projects
Crystal Maroon	80–200 people	Large projects
Crystal Diamond **or** Crystal Sapphire	Very large teams (200+ people)	Very large projects with high criticality

https://www.wrike.com/agile-guide/faq/what-is-agile-crystal-methodology/

Crystal Framework

When to Use Crystal or not

- Projects with small-to-medium teams
- When flexibility and team empowerment are needed
- Teams that prefer less documentation and more interaction
- Not ideal for highly regulated or safety-critical systems

SAFe (Scaled Agile Framework)

- A framework for applying Agile at enterprise scale
- Combines Agile, Lean, and DevOps principles
- Designed for large organizations with multiple teams working together
- Helps enterprises deliver value faster and more efficiently at scale.

SAFe (Scaled Agile Framework)

- Core Values:
  - Alignment
  - Built-in Quality
  - Transparency
  - Program Execution
- Principles include:
  - Apply systems thinking
  - Assume variability, preserve options
  - Build incrementally with fast, integrated learning cycles

SAFe (Scaled Agile Framework)

When to Use SAFe or not

- Large enterprises with many Agile teams
- Need for **strategic alignment** across business and tech
- Regulatory or high-dependency environments
- May be **too heavy** for small teams or startups
- *SAFe provides structure for scaling Agile — but requires strong leadership and training.*

Disciplined Agile (DA)

- A toolkit, not a framework — helps you choose your way of working (WoW)
- Blends Agile, Lean, Scrum, Kanban, SAFe, and even traditional methods
- Focuses on context-driven agility, not one-size-fits-all

Disciplined Agile (DA)

DA supports multiple life cycles, including

- Agile Lifecycle (Scrum-based)
- Lean Lifecycle (Kanban-based)
- Continuous Delivery (Agile or Lean flavor)
- Program Lifecycle (large-scale projects)
- Exploratory Lifecycle (for innovation)
- Waterfall Lifecycle (for traditional teams)

Disciplined Agile (DA)

When to Use DA or not

- Organizations seeking flexibility and guidance
- Teams that need help tailoring their WoW
- Enterprises blending multiple approaches
- Not ideal for teams wanting a prescriptive, rigid framework
- Disciplined Agile is about agility, tailor your approach for success.

Agile Leadership

Leading Effectively

- Servant Leadership
  - Leader provides what the team needs
    1. Shield team from interruptions
    2. Remove impediments to progress
    3. (Re)Communicate project vision
    4. Carry food and water

Twelve Principles for Leading Agile Projects

1. Learn the team members needs
2. Learn the project requirements
3. Act for the simultaneous welfare of the team and the project
4. Create an environment of functional accountability
5. Have a vision of the completed project
6. Use the project vision to drive your own behavior

©2002 Jeffery Pinto, Project Leadership from Theory to Practice

Twelve Principles for Leading Agile Projects

7. Serve as the central figure in successful project team development
8. Recognize team conflict as a positive step
9. Manage with an eye toward ethics
10. Remember that ethics is not an afterthought, but an integral part of our thinking
11. Take time to reflect on the project
12. Develop the trick of thinking backwards

Leadership Tools and Techniques

- Using these tools still need soft-skills approach
- Modeling Desired Behavior
  - Honesty
  - Forward-Looking
  - Competent
  - Inspiring
- Communicating project vision
- Enabling others to act
  - Switch from exclusive tools to inclusive tools
- Being willing to change the status quo

Leadership Task

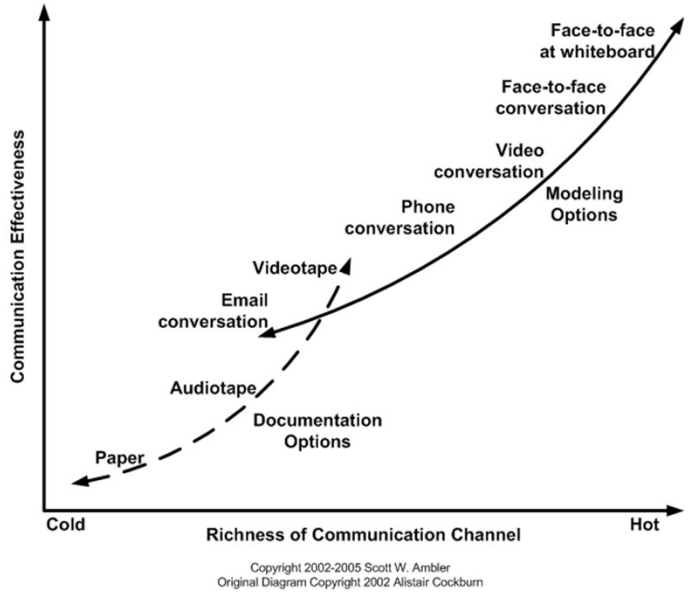
- Practice Transparency through Visualization
- Crate a safe environment for experimentation
- Experiment with new techniques and processes
- Share knowledge through collaboration

Communicating with Stakeholders

- Face to face communication
- Two-way communication
- Knowledge sharing
- Information Radiators
- Social Media

Face-to-face Communication

- Face to face communication



http://www.agilemodeling.com/essays/communication.htm

Communicating with Stakeholders

- Two-way communication
  - Just don't ask for confirmation or concerns, but actually listen to what they have to say
- Knowledge sharing
  - Agile teams work closely with each other such as with pair-programming.
  - Using Kanban boards or wireframes are ways to share information
  - Use of low-tech tools like a whiteboard will allow all to see the work and understand it
  - We must encourage it

Communicating with Stakeholders

- Information Radiators
  - Things that are highly visible
  - Used to display information
  - Usually includes chats, graphs and boards
- Social Media
  - Use to communicate
  - Can include twitter or Instagram

Tools for Agile Projects

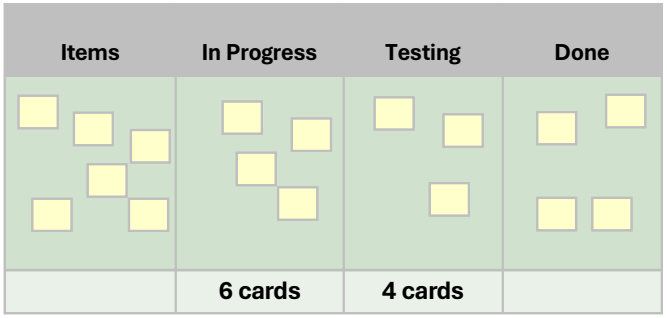
- Low-tech, high-touch over computer models
- When using computer models problems could arise such as:
  - Data accuracy perception increases
  - No stakeholder interaction. Only a few people would update them

Low-Tech, High-Touch Tools

- Use card, charts, whiteboards, and walls
- Promotes communication and collaboration
- Skip using a computer Gantt chart to a Kanban board

Kanban/Task Board

- An "information radiator" - ensures efficient diffusion of information
- Can be drawn on a whiteboard or even a section of wall
- Makes iteration backlog visible
- Serves as a focal point for the daily meeting



Educating People about Agile

- Teach all the stakeholders about the benefits of agile
- Concerns about agile can Include:
  - Senior management and sponsor: They are worried about the risk of failing
  - Managers: fear the loss of control
  - Project team: resist agile methods
  - Users: will not get all features

Engaging Stakeholders

- Short iterations and release keeps them engage
- Keeping then engage can lead to stakeholders being more involved and getting more change request
- This helps us to identify risk and issues early
- If some stakeholders are causing problems, the agile PM will need to use their interpersonal skills to resolve issues
- Need to have a process for escalating stakeholders issues
- Why such a big focus on stakeholders?
  - Projects and done by people for people

Methods of Stakeholder Engagement

- Get the right stakeholders
- Cement stakeholder involvement
- Actively manage stakeholder interest
- Frequently discuss what done looks like
- Show progress and capabilities
- Candidly discuss estimates and projections

Set a Shared Vision

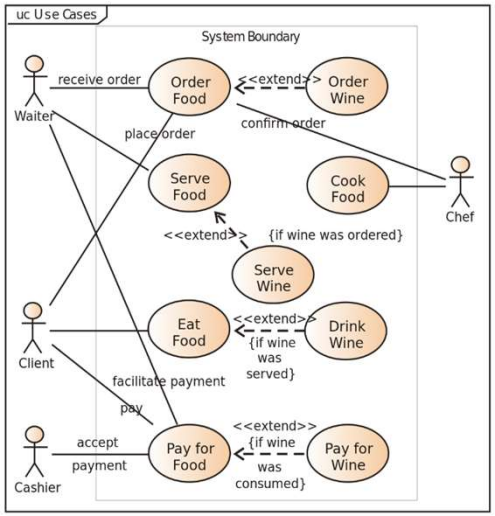
- Important to ensure customers and agile project team has the same vision
- Methods include:
  - Agile Charter
  - Definition of “Done”
  - Agile Modeling
    - Use case diagram
    - Data models
    - Screen design
  - Wireframes
  - Personas

Definition of “Done”

- Creating a shared vision of what done looks like
- Should be done for:
  - User stories
  - Releases
  - Final project deliverables

Agile Modeling

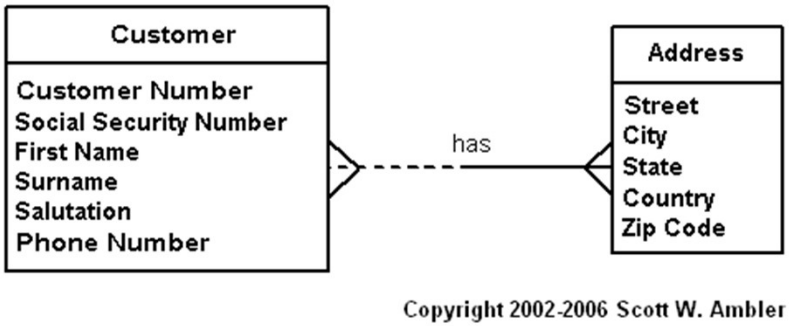
- Use case diagrams
  - Visually shows how users would use an application



https://en.wikipedia.org/wiki/Use_case_diagram

Agile Modeling

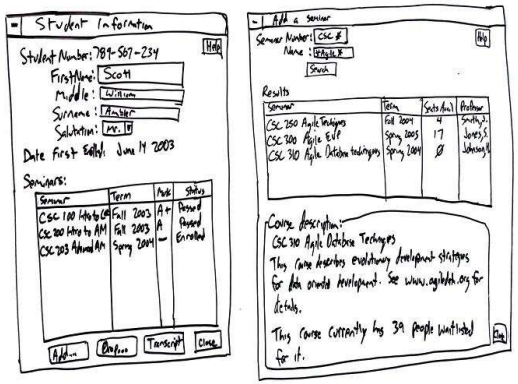
- Data models
  - How the data are structured in tables and their relationships



http://www.agiledata.org/essays/dataModeling101.html

Agile Modeling

- Screen designs
  - Simple screen shots



http://agilemodeling.com/artifacts/uiPrototype.htm

Personas

- Personas
  - Quick guides or reminders of key stakeholders and interests
    - Provide description of users
    - Be grounded in reality
    - Be goal-oriented, specific, and relevant
    - Be tangible and actionable
    - Generate focus
  - Help team focus on valuable features to users

Personas

Name: Andrew Jones– Certified Accountant



Description:
Andrew has been an Accountant for over 10 years and has worked at many large accounting firms.

He likes to be organized and get his work done on time.

Value:
Andrew would like to ensure all company bills are paid on time while using online auto payments.

He would like to ensure customers are reminded automatically of outstanding balances.

He is looking to print the receivables and payable reports on a weekly basics to check on bills and invoices.

Development / Delivery Team

- Group that build and test the increments of the product
  - Build product in increments
  - Update information radiators
  - Self organize and directing
  - Share progress by doing daily stand-up meetings
  - Demo the completed product increments
  - Holds retrospectives at the end of sprints
  - Does release and sprint planning and estimations

Product Owner/Customer

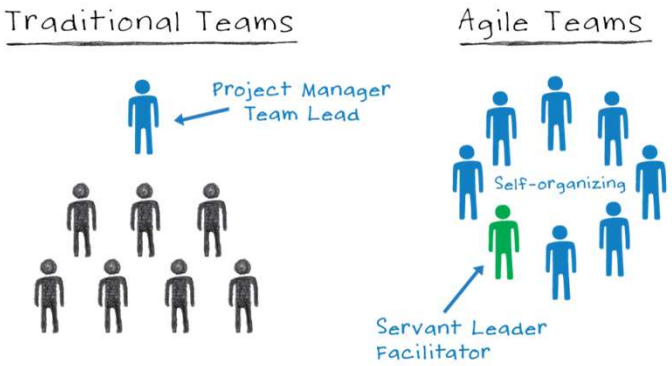
- Prioritizing the product features
- Manage the product backlog ensuing its accurate and up to date
- Ensures the team has a shared understanding of the backlog items
- Defines the acceptance criteria
- Provides the due dates for the releases
- Attends planning meeting, reviews, and the retrospective.

Agile Project Manager (ScrumMaster/Coach)

- Act as a servant leader
- Help the team self-organize and direct themselves
- Be a facilitator
- Ensure the team plan is visible and the progress is known to the stakeholders
- Act as a mentor and coach
- Work with the product owner to manage the product backlog
- Facilitates meeting
- Ensure issues are solved

Building Teams

- Self-Organizing
- Self-Directing
- Small teams with fewer than 12 members



Generalizing Specialists

- Have members that can do different tasks
- Members skilled in more than one area
- Share work reduce bottleneck

High-Performance Agile Teams

- Have a shared vision
- Realist goals
- Fewer than 12 members
- Have a sense of team identity
- Provide strong leadership

Experiments (Have a safe place)

- Establish safe environment for disagreement
- Allows team members to build strong commitment to decisions
- Encourage people to experiments with new methods
- Leads to more engagement

Welcome Constructive Disagreement

- Leads to better buy-in and decisions
- Avoiding conflicts can lead to conflicts escalating
- A safe place for disagreement leads to successful problem solving

Team Spaces

- Co-located Teams
- Team Spaces
- Osmotic Communication
- Global and Cultural Diversity
- Distributed teams

Co-Located Teams

- All team member work together in the same location
- Allows for face-to-face time and interaction
- Should be within 33 feet of each other
- No physical barriers
- Sometimes a virtual co-location

Team Space

- Lots of low-tech, high touch
  - Whiteboards and task boards
  - Sticky notes, flip charts
  - Round table
  - No barriers to face-to-face communication
- Caves and Common
  - Caves → space team members can retreat to individually
  - Common → space team members can work as group
- Osmotic Communication
  - Information flows that occur as part of everyday conversations and questions
  - 33 feet or 10 meters
- Tacit Knowledge
  - Information that is not written down; supported through collective group knowledge

Global and Cultural Diversity

- Time Zones
- Cultures
- Native Languages
- Styles of communications

Distributed Teams

- At least one team member working off-site
- Need to find ways to replicate co-location team benefits
- Agile Tools
  - Low-Tech, High-Touch Tools
  - Digital Tools for distribute teams
    - Video conferencing
    - Interactive whiteboards
    - IM / VoIP
    - Virtual card walls
    - Web cams
    - Digital cams

Models of team development

- Shu-Ha-Ri Model of Skill Mastery
  - Shu- Obey,
  - Ha – Moving away,
  - Ri – finding individual paths
- Dreyfus Model of Adult Skill Acquisition
  - Novice, Advanced Beginner, Competent, Proficient, Expert

Tuckman's Five Stages of Team Development

1. **Forming**: team comes together and starts to get to know each other. There is not much conflict or communication.
2. **Storming**: team members start to have conflicts with each other. They start to learn of each other's ideas and may not agree with them. Most conflicts takes place in this stage.
3. **Norming**: the team members begin to agree with each other on the best methods to build the deliverables. Generally, everyone is coming to a consensus.
4. **Performing:** the team is performing well and is working without conflict.
5. **Adjourning**: In this stage, the project is completed and the team is reassigned.

Solving Problems

- Problem Solving as continuous improvement
- Engage the team
- Some problems can’t be solved

Why Engaging the Team?

- Team usually produces the best practical solutions
- Benefits
  - Get consensus from all members
  - Gets a broad knowledge base
  - Solutions are practical
  - When ask people work hard to produce good ideas
  - Asking someone for help shows confidence
- Usage and Cautions
  - Solve real problems
  - Poor team cohesion
  - Team and project changes
  - Follow-Through

Verifying and Validating Value

- “Gulf of Evaluation”
  - What one person describes is often different from how another interprets

Frequent Verification and Validation

- Resolve problems as soon as possible
- Don’t let little problems grow over time



Negotiation

- This happens all throughout the project
- Good negotiation will allow everyone to investigate the options and trade-offs
- Most effective when interactions between people are positive and there are room for give and take

Using Workshops

- Meeting when work gets done
- Retrospectives are a type of workshops
- Ways to make them more effective:
  - Diverse groups has a larger perspective
  - Use methods such as round-robin to ensure no one dominates
  - Try to get everyone to participate in the first few minutes
- User story workshops are where we write the user stories and keep stakeholders engage

Active Listening

- Level 1: Internal – how is it going to affect me
- Level 2: Focused – put ourselves in the mind of the speaker
- Level 3: Global – builds on level with body language

Brainstorming

- Brainstorming
  - Quite Writing
    - Give people about 5 minutes to write down their ideas
  - Round-Robin
    - Pass a token around to ensure everyone will speak
  - Free-for-all
    - People shout out their taught. May only work in a supportive environment

Facilitation

- Run effective meeting and workshops.
- Have the following:
  - Goals
  - Rules
  - Timing
  - Assisting

Failure Modes

Why do people Fail:

1. Making mistakes
2. Preferring to fail conservatively
3. Inventing rather than researching
4. Being creatures of habit
5. Being inconsistent

Success Modes

- Why do we succeed:
  1. Being good at looking around
  2. Being able to learn
  3. Being malleable
  4. Taking pride in work

Success Strategies

- Balance discipline with tolerance
- Start with something concrete and tangible
- Copy and alter
- Watch and listen
- Support both concentration and communication
- Match work assignment with the person
- Retain the best talent
- Use rewards that preserve joy and combine rewards
- Get feedback

Green Zone/Red Zone

- Red Zone:
  - Blames others for everything
  - Responds defensively
  - Feels threatened
  - Triggers defensiveness
  - Doesn't let go or forgive
  - Uses shame and blame
  - Focus on short-term advantage
  - Doesn't seek or value feedback
  - Sees conflict as a battle and only seeks to win
  - Communicates high level of disapproval
  - Sees others as the problem or enemy
  - Does not listen effectively

Green Zone/Red Zone

Green Zone:

- Take responsibility
- Seeks to respond nondefensively
- Is not easily threatened psychologically
- Attempts to build success
- Uses persuasion rather than force
- Thinks both short and long term
- Welcomes feedback
- Sees conflict as a natural part of life
- Seeks excellence rather than victory
- Listens well

Conflict Resolution

- All projects will have conflicts
- While some level of conflicts are good, we need to ensure they don't become a "world war" where people are trying to destroy each other
- Levels of conflict(1-5):
  - Level 1: Problem to solve – sharing info
  - Level 2: Disagreement – Personal Protection
  - Level 3: Contest – Must win
  - Level 4: Crusade – Protecting one's group
  - Level 5: World War – Must destroy the other

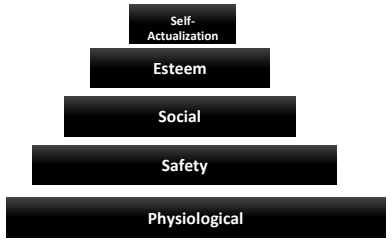
Participatory Decision Models

- Engage stakeholder in decision making process
  - Simple voting
    - Vote “for” or “against” it
  - Thumps up/down/sideways
    - People hold their thumps in a way of if the support it or not. Sideway is if they cannot make up their mind
  - Fist of five
    - People how up finger based on they support the idea
    - 1 finger: total support – 5 finger: Stop against it

Theory of Motivation

- Recognition and Rewards
  - Rewarding good behavior, Only desirable behavior should be rewarded, used to increase morale
  - Maslow's hierarchy of needs

1. Physiological: The necessities to live: air, water, food, clothing, and shelter.
2. Safety: People need safety and security; this can include stability in life, work, and culture.
3. Social: People are social creatures and need love, approval, and friends.
4. Esteem: People strive for the respect, appreciation, and approval of others.
5. Self-actualization: At the pinnacle of needs, people seek personal growth, knowledge, and fulfillment.



Theory of Motivation

- Herzberg's Theory of Motivation
  - Hygiene agents (What factors influence satisfaction at work) are expected by and can only demotivate if they are not present. Motivating agents provide opportunity to exceed, and advance.
- McGregor's Theory X and Y
  - Theory X- is bad. These people need to be watched all the time, micromanaged, and distrusted, people avoid work, responsibility, and have no ability to achieve.
  - Theory Y is good. These people are self-led, motivated, and can accomplish new tasks proactively.
- Theory Z
  - Increased Loyalty at the workplace. Theory emphasizes the well-being of the employees, both at work and outside of work, it encourages steady employment

Theory of Motivation

- Expectancy Theory
  - People behave based on what they expect as a result of their behavior.
- McClelland 3 need theory
  - Achievement
  - Power
  - Affiliation
- Forms of Power
  - Reward Power - Ability to give rewards
  - Expert Power - SME
  - Legitimate(formal power)
  - Referent- Respect /Personality of the Manger
  - Punishment- Punish associates when they fail (least desirable)